MSc in Foundations of Data Science University of Hertfordshire

Dr. Yi Sun

Practical Session 2

1. Aims of the Session

In this session, you will learn 1) more on how to do matrix operations using **Numpy**; 2) more on computation on arrays using **Numpy**; 3) how to define your own functions in Python; and 4) more data types in Python.

2. Starting with **Jupyter Notebook** for **Python**

In this module, you will use the syntax of Python 3. There are many options for development environments for Python. You will use the Jupyter notebook, which is a browser-based graphical interface to the IPython (*Interactive Python*) shell. On windows,

- click the **window icon** on the bottom left corner.
- search **Jupyter Notebook** by typing **Jupyter** from the search box under All Programs.
- click on **Jupyter Notebook.**
- click **New** from **Jupyter Notebook**, then select **Python 3 (ipykernel)** from the pop-up list. You should see a new page with a blank cell, where you can type in Python code.
- click **File** and select **Rename** from the pop-up list, you may give the file a name you prefer, for example, *practical*2.

3. More using matrices in Python

3.1 More basic operations:

Again, as you have learnt in Practical1, you will use built-in functions in **NumPy** to do basic computation.

Type the code in the following code snippet into your **Jupyter** notebook and then click the '**run**' button.

```
import numpy as np

M = np.array([[2,3],[5,-8]])
N = np.array([[1,-4], [8, -6]])

print('M+N=', M+N)
print('M*N=', M*N)
print('M@N=', M@N)
print('M.dot(N)=', M.dot(N))

print('The transpose of M is:',np.transpose(M))
```

As you will see from the results, '*' returns the product of two matrices, element-wise; '@' and '**.dot()**' are used for matrix multiplication. The **transpose** function is used to give the matrix transpose.

3.2 numpy.linalg

© 2019, 2022 University of Hertfordshire Higher Education Corporation

numpy.linalg is a class having a standard set of things like inverse, determinant and matrix decompositions, You need to import **numpy.linalg** first:

```
import numpy.linalg as LA
```

Task1:

1) ***compute the inverse of the matrix M defined in Subsection 3.1 using LA.inv(M).**
2) ***compute the determinant of matrix N defined in Subsection 3.1 using LA.det(N).**
3) define a matrix $A = \begin{bmatrix} 3 & 1 & 2 \\ 1 & 4 & 0 \\ 2 & 0 & 5 \end{bmatrix}$, and then do the eigendecomposition as shown as follows:

```
eignVal, eignVec = LA.eig(A)
```

where **eignVal** includes eigenvalues and **eignVec** is the corresponding eignvectors. Details can be viewed here: https://numpy.org/doc/stable/reference/generated/numpy.linalg.eig.html

Further, if you run the following code, what can you find? Can you think of anything special about the matrix A?

```
np.dot(eignVec, eignVec.T)
```

Note that the way you call each function can be different depending on how you import the **numpy.linalg** module, that is, with or without an alias. More details can be found here:

https://docs.scipy.org/doc/numpy/reference/routines.linalg.html

4. More about computation on **Numpy** Arrays

4.1 Comparisons and Boolean logic

Generate a new array **x3** by typing **x3 =np.arange(15)** first.

The table on the next page includes three Boolean (true/false statements) operators and their equivalent ufuncs in the first two columns separately. Using these tools, you can manipulate values in an array based on some criterion. For example, to get the sum of elements whose value is either greater than 10 or less than 3, you can type the following:

```
sx3= np.sum(x3[(x3>10) | (x3<3)])
```

Task2: Your task is to understand each operator listed in the table by working on the examples given in the third column. That is to type each command in your Jupyter notebook, run it and then understand the corresponding result.

© 2019, 2022 University of Hertfordshire Higher Education Corporation

Operator	Equivalent ufunc	Typing command
&	np.bitwise_and	a=x3[(x3<=10) & (x3>=3)] a1=np.bitwise_and((x3<=10),(x3>=3))
|	np.bitwise_or	b= x3[(x3>10) | (x3<3)] b1 = np.bitwise_or(x3>10, x3<3)
~	np.bitwise_not	c = x3[~(x3>5)] c1 = np.bitwise_not(x3>5)

4.2 Selecting random points

Sometimes you need to randomly select a number of data points from a dataset. Generate an array as follows:

```
rand=np.random.RandomState(421)

rx = rand.randint(50, size=10)
print(rx)
```

```
[23 44 19 17 15 30 38 17 47 32]
```

Suppose we want to randomly select 5 data points from this array. You can type the following code:

```
indices = np.random.choice(rx.shape[0], 5, replace=False)
indices
```

```
array([5, 2, 8, 4, 7])
```

```
selected = rx[indices]
print(selected)
```

```
[30 19 47 15 17]
```

Task3: Modify the value of those 5 selected elements to zero.

4.3 Sorting arrays

Although Python has built-in **sort** and **sorted** functions to work with lists, **NumPy** provides **np.sort** and **np.argsort**, which are much more efficient and useful due to using a *quick-sort* algorithm.

np.sort: return a sorted version of the array.

np.argsort: return the indices of the sorted elements.

Task4: investigate how to use these two functions by using the help function first. Then apply these two functions on **x4=[10, 9, -1, 3].** Further, can you sort **x4** in descending order?

5. Define your own functions

You can define your own functions in Python and reuse them throughout the program. The syntax for defining a function is as follows:

© 2019, 2022 University of Hertfordshire Higher Education Corporation

def NameofFunction (list of parameters) :

code detailing what the function does

return [expression]

There are two key keywords here: def and return. def tells the program that the ***indented*** code from the next line onwards is part of the function. return is the keyword that to return an answer from the function. Note that Python uses indentation.

For example, you need to define a function to add two matrices, both of which have a size of 2 by 2. Remember that you can add only the corresponding elements in two matrices. The function is shown as follows, and you need to type it in your own Jupyter Notebook.

```
def addMatrices(a,b):
    '''adds two 2x2 matrices together'''
    C = [[a[0][0]+b[0][0], a[0][1]+b[0][1]],
         [a[1][0]+b[1][0], a[1][1]+b[1][1]]]
    return C
```

Now suppose that you want to add two matrices using the function **addMatrices**:

A=[[1, 2], [8, -3]]

B=[[0, 10],[-2, 6]]

You need to type in these two matrices first, then type **C = addMatrices(A, B)**. To see the result, you may type **print(C)**.

Task5: Define a function which can add values along the diagonal line of a given 3 by 3 matrix. Report the result for matrix **M = [[1, 2, 3], [0, 10, -5], [-6, -2, 11]]** by using the function you have defined.

6. Accessing source code with **??**

Create a function, which can compute the square of a number as follows:

```
def square(x):
    """Return the square of x"""

    return x ** 2
```

After typing you need to click the '**Run**' button . Now type **square?** in a new cell. What information have you got?

IPython provides a shortcut to the source code with the double question mark (**??**). For example, type **square??** to see what you get.

Sometimes the ?? suffix does not display any source code. For example, you may type **print??** to see what you get. This is because the object in question is not implemented in **Python**, but in **C** or some other compiled extension language.

© 2019, 2022 University of Hertfordshire Higher Education Corporation

7. More about Data Types in Python

Data type simply refers to the type of data that a variable stores. Apart from integer and float you have seen in the previous session, **Python** provides more types. In this session, you will learn three more: the string, list, and dictionary.

- String

  To declare a string, you can use either single quotes or double quotes as shown as follows:

```
firstName ='David'
surname = "Lee"
```

  You can combine multiple substrings by using the plus sign (+). For instance:

```
username = firstName + " " + surname
```

```
username
```

```
'David Lee'
```

- List

  List refers to a collection of data which are normally related. Square brackets are used when declaring a list. Multiple values are separated by a comma. List elements can be of different data types. For example,

```
myList= [1, 2, 6, 10, 'Hello']
```

  New elements can be added either to the end of a list using the built-in function **append()** or to a list at a particular position using the built-in function **insert()**. For example, *myList.insert(specified position, new element)*

  Task6: Add a string 'e' to **myList** at the second position; add **0** to the end of the list.

- Dictionary

  Dictionary is a collection of related data pairs. For instance, if we want to store the name and score of 5 students, we can store them in a dictionary.

  To declare a dictionary, you write dictionaryName = {dictionary key : data}. For example:

```
myDictionary ={"Peter":80, "Mary":59, "Ann":81, "David":50, "Joe":62}
```

```
myDictionary
```

```
{'Peter': 80, 'Mary': 59, 'Ann': 81, 'David': 50, 'Joe': 62}
```

© 2019, 2022 University of Hertfordshire Higher Education Corporation

Note that the dictionary keys must be unique within one dictionary. The following example shows that "Peter" as the dictionary key has been used twice:

```
myDictionary ={"Peter":80, "Mary":59, "Ann":81, "David":50, "Peter":62}
```

```
myDictionary
```

```
{'Peter': 62, 'Mary': 59, 'Ann': 81, 'David': 50}
```

To access individual items in the dictionary, you can use the dictionary key. For instance, to get Ann's score, you write *myDictionary["Ann"]*

References

1. J. VanderPlas: Python Data Science Handbook, 2016, O'Reilly Media, Inc.

Practical Exercise Information:

The practical exercise for sub-tasks 1) and 2) in Task 1 of this handout will take place in your timetabled practical session on **22/10/2025**. Attendance at the timetabled practical sessions, as shown in your personal timetable, is compulsory. Marks will only be awarded to students who both submit the required code and sign the attendance sheet. **Please note that short extensions are not permitted for this coursework.**

In this exercise, you will complete similar questions using new data and must submit your code as both a **Jupyter Notebook** and an **HTML file** to Canvas.

This exercise is one of the four programming exercises that make up Coursework Part 1 – Practical Exercises**,** and it is worth 3 marks**.**

- **3 marks**: Correct code and results are provided in both the Jupyter Notebook and its exported HTML file.
- **2 marks**: Minor mistakes are presented in the code.
- **1 mark**: Major mistakes are presented in the code.
- **0 marks**: No attempt.

The best three out of four scores will be counted towards Coursework Part 1 -Practical Exercises, which is worth 10% of the final grade.

The following applies to this assignment in relation to the use of Generative AI:

In this assessment the student is permitted to use GenAI tools (or a proofreader or proofreading service) to proofread their work but not permitted to use AI tools in the creation of content for their work.

© 2019, 2022 University of Hertfordshire Higher Education Corporation